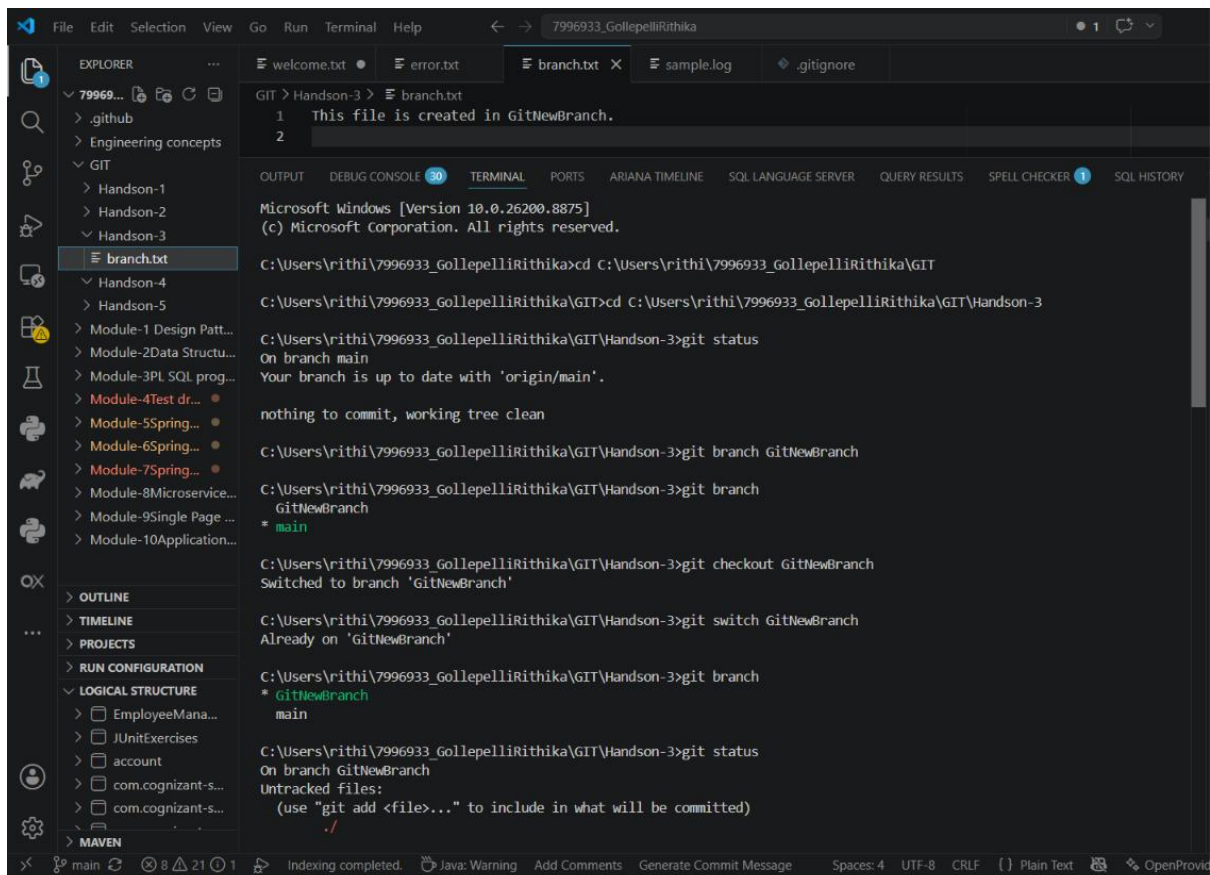


Output3-Handson3-GIT.HOL



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left displaying a file tree. The main editor area shows a terminal window with the following commands and output:

```
Git > Handson-3 > branch.txt
1 This file is created in GitNewBranch.
2

Microsoft Windows [Version 10.0.26200.8875]
(c) Microsoft Corporation. All rights reserved.

C:\Users\rithi\7996933_GollepelliRithika>cd C:\Users\rithi\7996933_GollepelliRithika\GIT

C:\Users\rithi\7996933_GollepelliRithika\GIT>cd C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git branch GitNewBranch

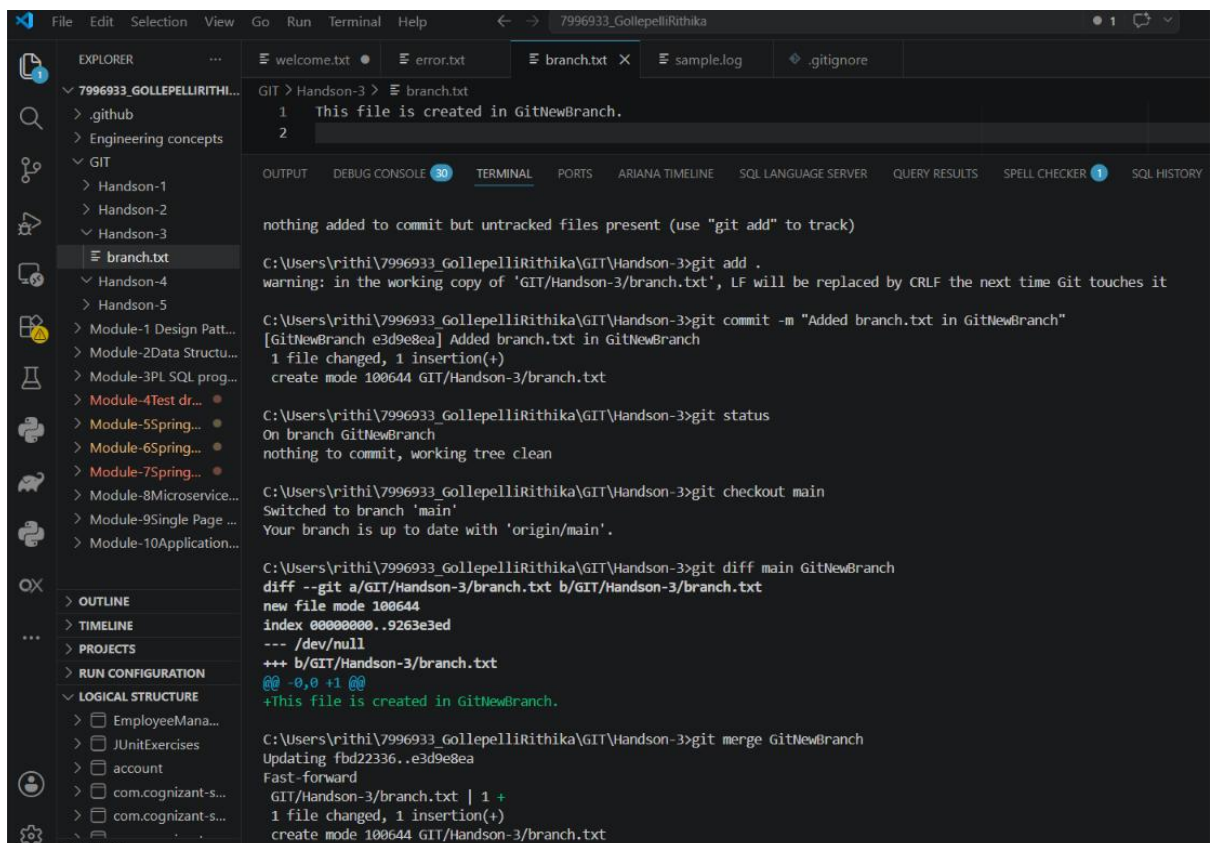
C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git branch
GitNewBranch
* main

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git checkout GitNewBranch
Switched to branch 'GitNewBranch'

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git switch GitNewBranch
Already on 'GitNewBranch'

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git branch
* GitNewBranch
  main

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git status
On branch GitNewBranch
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  ./
```



The screenshot shows the Visual Studio Code interface with the Explorer sidebar on the left displaying a file tree. The main editor area shows a terminal window with the following commands and output:

```
Git > Handson-3 > branch.txt
1 This file is created in GitNewBranch.
2

nothing added to commit but untracked files present (use "git add" to track)

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git add .
warning: in the working copy of 'GIT/Handson-3/branch.txt', LF will be replaced by CRLF the next time Git touches it

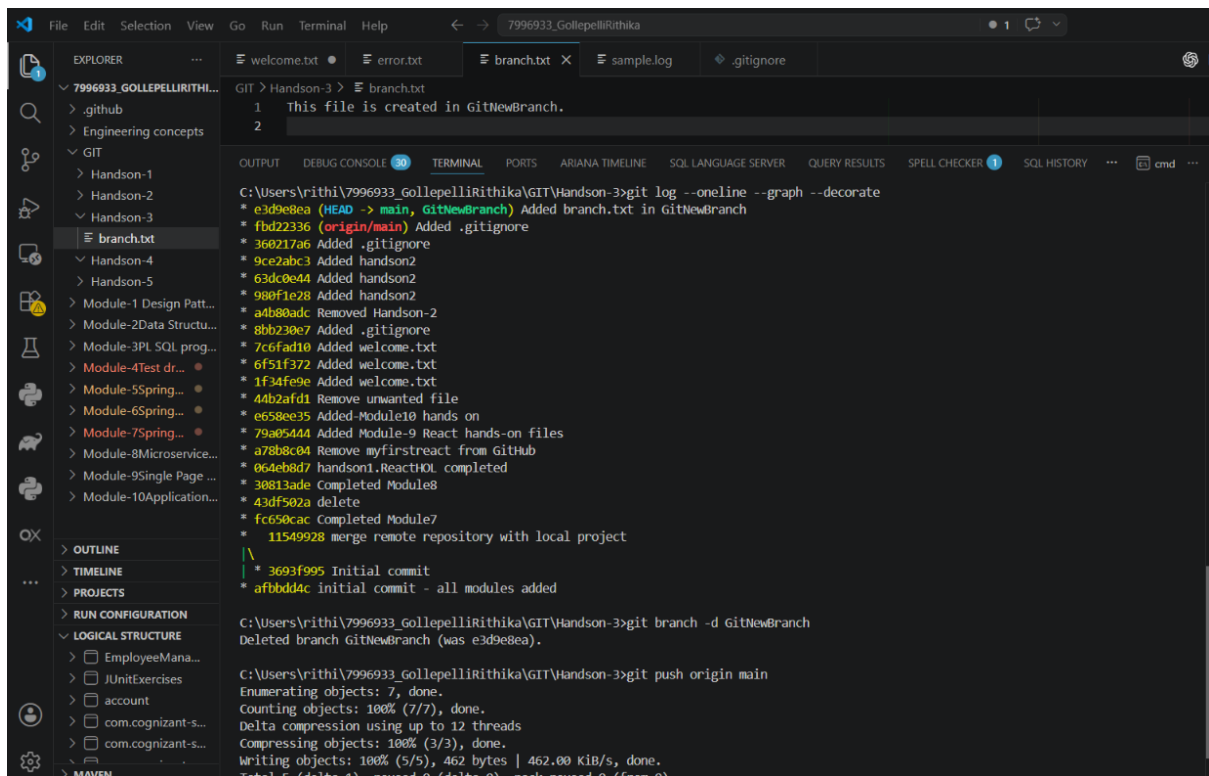
C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git commit -m "Added branch.txt in GitNewBranch"
[GitNewBranch e3d9e8ea] Added branch.txt in GitNewBranch
1 file changed, 1 insertion(+)
create mode 100644 GIT/Handson-3/branch.txt

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git status
On branch GitNewBranch
nothing to commit, working tree clean

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git checkout main
Switched to branch 'main'
Your branch is up to date with 'origin/main'.

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git diff main GitNewBranch
diff --git a/GIT/Handson-3/branch.txt b/GIT/Handson-3/branch.txt
new file mode 100644
index 00000000..9263e3ed
--- /dev/null
+++ b/GIT/Handson-3/branch.txt
@@ -0,0 +1 @@
+This file is created in GitNewBranch.

C:\Users\rithi\7996933_GollepelliRithika\GIT\Handson-3>git merge GitNewBranch
Updating fbd22336..e3d9e8ea
Fast-forward
 GIT/Handson-3/branch.txt | 1 +
 1 file changed, 1 insertion(+)
create mode 100644 GIT/Handson-3/branch.txt
```



1. Explain Branching and Merging

What is Branching

A branch is a separate copy of the project's code where developers can work on new features, fix bugs, or experiment without affecting the main codebase.

- The default branch is usually called main or master.
- Each developer can create their own branch to work independently.
- Changes made in one branch do not affect other branches until they are merged.

Why Use Branches?

- Develop new features safely.
- Fix bugs without disturbing the main project.
- Allow multiple developers to work simultaneously.

Common Git Branch Commands

Create a new branch:

`git branch feature-login`

Switch to the branch:

`git checkout feature-login`

Or create and switch together:

`git checkout -b feature-login`

View all branches:

```
git branch
```

What is Merging

Merging combines the changes from one branch into another branch.

For example:

- A developer completes work in the feature-login branch.
- The changes are merged into the main branch.

Steps to Merge a Branch

Switch to the main branch:

```
git checkout main
```

Merge the feature branch:

```
git merge feature-login
```

If there are no conflicts, Git automatically combines the changes.

Merge Conflicts

A merge conflict occurs when two branches modify the same part of a file differently.

To resolve a conflict:

1. Open the conflicted file.
2. Edit the file and keep the correct code.
3. Save the file.
4. Stage the changes:

```
git add .
```

5. Complete the merge:

```
git commit
```

Advantages of Branching and Merging

- Enables parallel development.
- Protects the main branch.

2. Explain About Creating a Branch Request in GitLab

What is a Branch in GitLab

A branch in GitLab is a separate version of the project used to develop new features or fix issues without affecting the main branch.

Steps to Create a Branch in GitLab

1. Log in to GitLab.
2. Open your project.

3. Select Repository - Branches.
4. Click New Branch.
5. Enter a branch name (e.g., feature-login).
6. Select the source branch (usually main).
7. Click Create Branch.

Alternatively, create a branch locally and push it:

```
git checkout -b feature-login
```

```
git push -u origin feature-login
```

After pushing, the new branch appears in GitLab.

Benefits of Creating Branches

- Isolates new work.
- Prevents accidental changes to the main branch.
- Allows multiple developers to work simultaneously.